R3.04 – Qualité de développement
Patrons de conception

Pierre Pomeret-Coquot
pierre.pomeret@iut-rodez.fr

Université Toulouse Capitole – IUT de Rodez

2026 – 2027







Plan du cours

1. Introduction
2. Patrons de conception du *Gang of Four*
   - Singleton
   - Adapteur
   - Façade
   - Stratégie
   - Composite
   - Itérateur
   - Proxy
   - Décorateur
   - Fabrique simple
   - Observateur
3. Références

CC BY

Origine du terme

Patrons de conception – terme repris de l'architecture

- Un **problème** se produisant fréquemment
- Une **solution** à ce problème **utilisable** (à **adapter** à chaque situation)

Exemple : #112 : Créer une transition d'entrée

Buildings, and especially houses, with a graceful transition between the street and the inside, are more tranquil than those which open directly off the street.

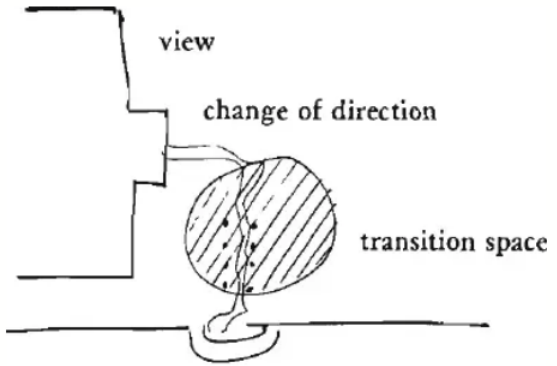


An abrupt entrance—no transition.



Each creates the transition with a different combination of elements.

Make a transition space between the street and the front door. Bring the path which connects street and entrance through this transition space, and mark it with a change of light, a change of sound, a change of direction, a change of surface, a change of level, perhaps by gateways which make a change of enclosure, and above all with a change of view.



A Pattern Language : Towns, Buildings, Construction – C. Alexander, 1977

CC BY

Patrons (informatique)

→ Décrire des **schémas de solution** récurrents à des **problèmes courants** dans des **situations**.

- Une abstraction d'un doublet, triplet ou d'un ensemble de classes qui peut être réutilisé encore et encore pour le développement d'applications (Coad 1992)
- Une règle tripartite exprimant une relation entre un certain contexte, un certain problème qui apparaît répétitivement dans ce contexte et une certaine configuration logicielle qui permet la résolution de ce problème (Appleton 1997)
- Un groupe d'objets coopérants liés par des relations et des règles qui expriment les liens entre un contexte, un problème de conception et sa solution (Aarsten 1996)

CC BY

Patrons : classification

- **Architectural patterns** : Schémas d'organisation structurelle de logiciels (ex : MVC, pipes, etc.)
- **Design patterns** : Structure de conception commune à plusieurs applications (cf. suite du cours)
- **Idioms pattern / cooding patterns** : Solution liée à un langage particulier (ex : simuler l'héritage multiple en Java)
- **Anti-patterns** : Mauvaise solution / comment s'en sortir
- **Organizational patterns** : Pattern "humain" pour le développement logiciel.

CC BY

Patrons de conception : classification

Catégories :

- **Création** : Patrons décrivant la façon d'instancier un ou plusieurs objets
  → Isoler le code relatif à l'initialisation
- **Structure** : Patrons décrivant les connections entre objets
  → Rendre ces connections indépendantes de l'application
- **Comportement** : Patrons décrivant les interactions entre objets
  → Gestion dynamique des interactions entres classes et/ou objets

Portée :

- **Classes** : relations entre classes, réutilisation par héritage
- **Objets** : relations entre objets, réutilisation par composition

CC BY

Patrons de conception du *Gang of Four*

	Création	**Structure**	**Comportement**
Classe	Fabrique simple	Adaptateur	Patron de méthode
Objet	Fabrique abstraite	Pont	Chaîne de responsabilités
	Prototype	Composite	Commande
	Singleton	Décorateur	Itérateur
	Monteur	Façade	Observateur
		Proxy	État
		Poids-Mouche	Stratégie
			Visiteur
			Interpréteur
			Médiateur
			Memento

CC BY

Patron de conception « Singleton »

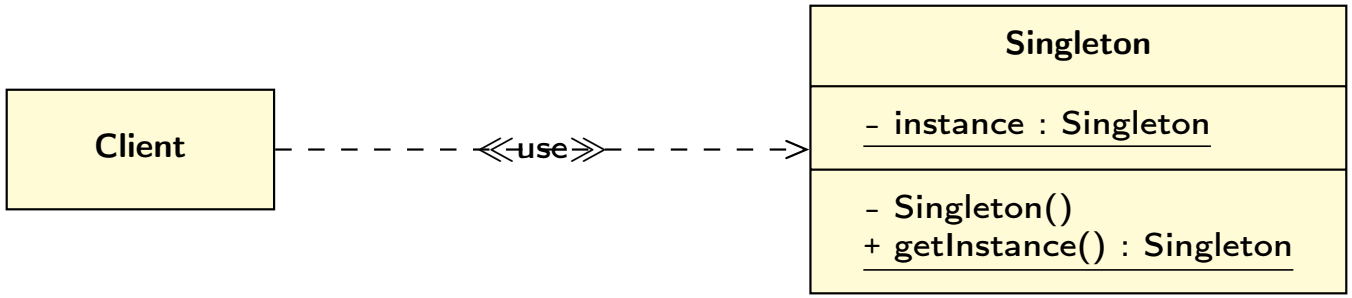
S'assurer qu'une classe n'a qu'une seule instance et fournir cette instance globalement.

- **Problématique :** Différentes classes / utilisateurs doivent se référer à la même instance unique.
  Sans unicité, on s'expose à des problèmes de fonctionnement.
- **Solution :** Garantir l'unicité de l'instance en contrôlant le constructeur.
- **Exemples :** Serveur d'impression, gestionnaire de fenêtre, lecture / écriture disque, DB, etc.

(rôle = création ; portée = objet)

CC BY

« Singleton » en Java




```java
public final class Singleton {

    private static Singleton instance = new Singleton();

    private Singleton() { /* code */ }

    public static Singleton getInstance() {
        return this.instance;
    }
}
```

```java
// Quelque part dans le code
Singleton instance = Singleton.getInstance();
faireUnTruc(instance);
```

```java
// Ailleurs dans le code
Singleton instance = Singleton.getInstance();
faireAutreChose(instance);
```

CC BY

Patron de conception « Adapteur »

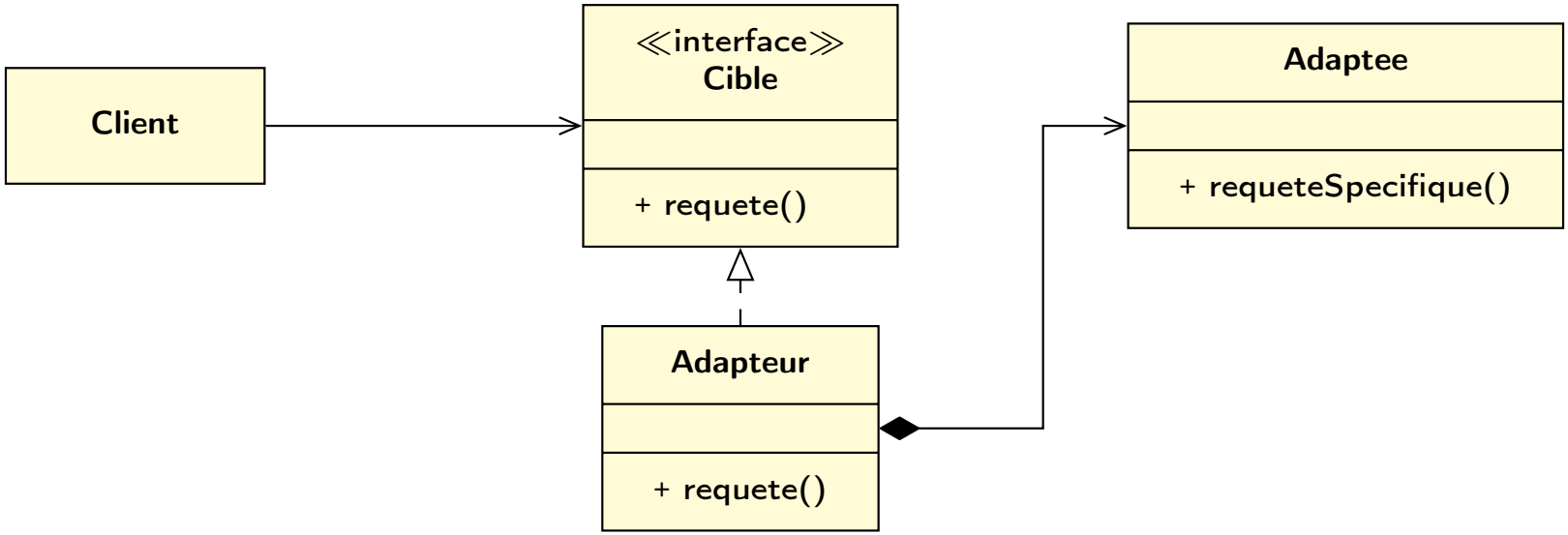
Faire correspondre une classe à une interface.

- **Problématique :** On veut utiliser une classe existante n'implémentant pas l'interface voulue.
- **Solution :** Créer une nouvelle classe implémentant l'interface et reposant sur la classe existante
- **Exemples :** Adaptateur HDMI/VGA, adaptateur prises françaises/anglaises, etc.

(rôle = structure ; portée = classe)

CC BY

« Adapteur » en Java




```java
public interface Cible {
    public int requete();
}
```

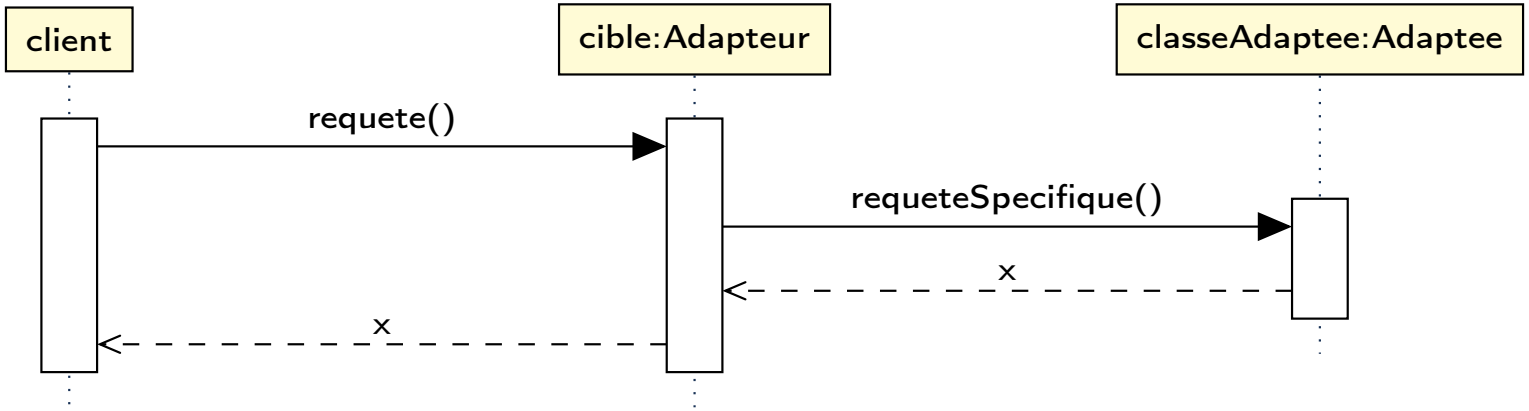
```java
public class Adapteur implements Cible {
    private Adaptee classeAdaptee = new Adaptee();

    public int requete() {
        return classeAdaptee.requeteSpecifique();
    }
}
```

```java
// Quelque part dans le code
Cible cible = new Adapteur();
int i = cible.requete();
```

CC BY

« Adapteur » – Diagramme de séquence




```java
public interface Cible {
    public int requete();
}
```

```java
// Quelque part dans le code
Cible cible = new Adapteur();
int i = cible.requete();
```

```java
public class Adapteur implements Cible {
    private Adaptee classeAdaptee = new Adaptee();

    public int requete() {
        return classeAdaptee.requeteSpecifique();
    }
}
```

CC BY

Patron de conception « Façade »

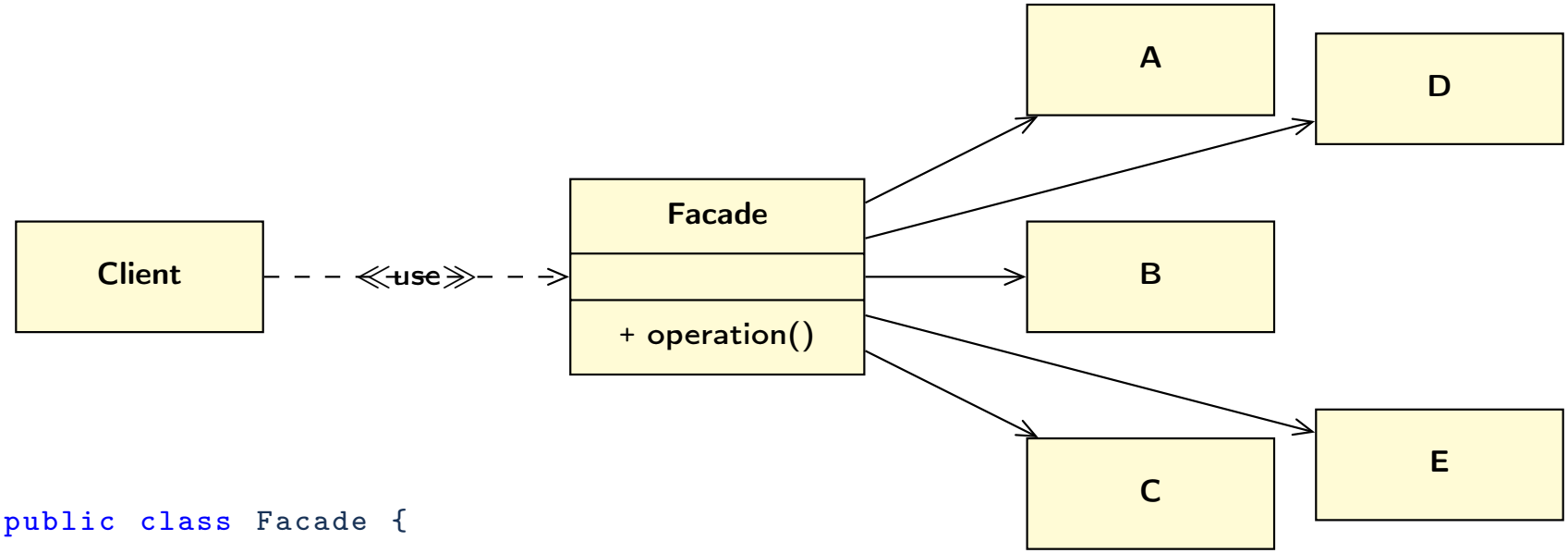
Cacher / découpler un sous-système et en fournir une interface simple et unifiée

- **Problématique :** On veut disposer d'une interface simple masquant un sous-système.
  On n'utilise pas tout le sous-système.
- **Solution :** Unifier les services dans une classe qui combine les services du sous-système
- **Exemples :** Fenêtres prédéfinies de Java Swing

(rôle = structure ; portée = objet)

CC BY

« Façade » – En Java




```java
public class Facade {

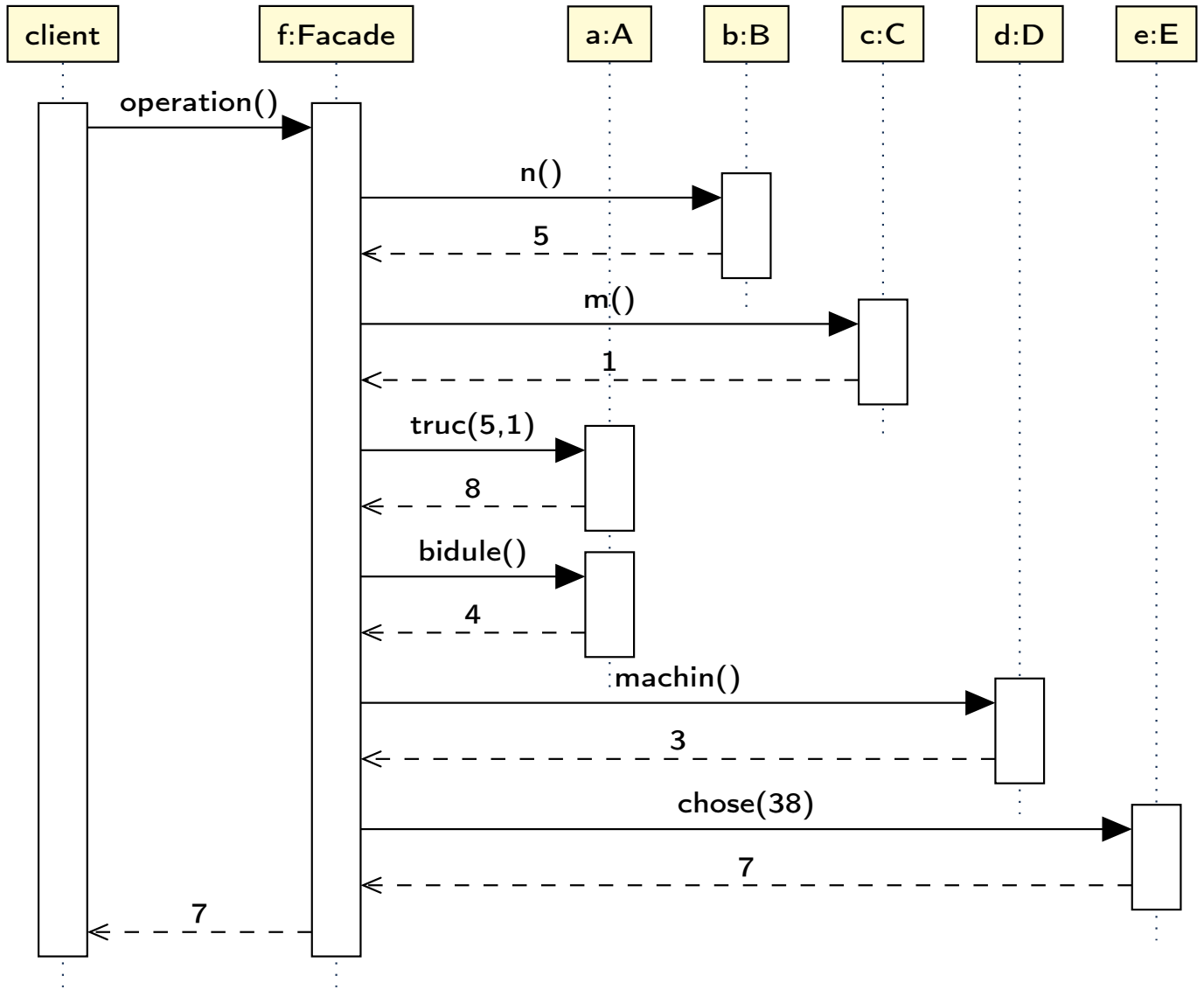
    private A a = new A();
    private B b = new B();
    // etc.

    public int operation() {
        int valeur;
        valeur = a.truc(b.n(), c.m());
        valeur = valeur + a.bibule();
        valeur = valeur * d.machin();
        return e.chose(valeur);
    }
}
```

```java
// Quelque part dans le code
Facade facade = new Facade();
int resultat = facade.operation();
```

CC BY

« Façade » – Diagramme de séquence



CC BY

Patron de conception « Stratégie »

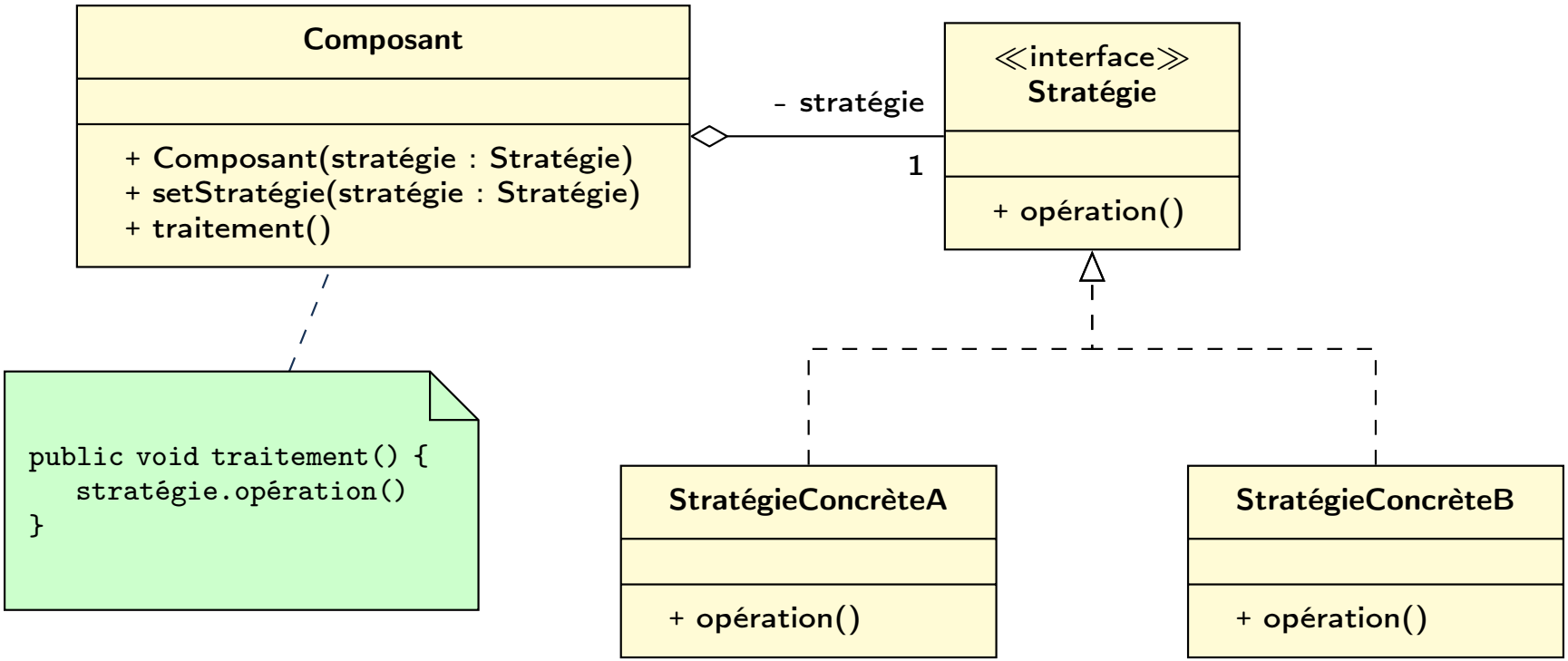
Rendre interchangeable une famille d'algorithmes / de services.

- **Problématique :** On souhaite que le client puisse sélectionner l'algorithme qui lui convient
- **Solution :** Séparer la sélection de l'algorithme et son implémentation.
- **Exemples :** Choisir la manière de distribuer les cartes ; choisir un moyen de transport, etc.

(rôle = comportement ; portée = objet)

CC BY

« Stratégie »



CC BY

« Stratégie » – En Java

```java
interface Strategie {
    void operation();
}
```

```java
class StrategieConcreteA implements Strategie {
    public void operation() {
        //Traitement A
    }
}
```

```java
class StrategieConcreteB implements Strategie {
    public void operation() {
        //Traitement B
    }
}
```

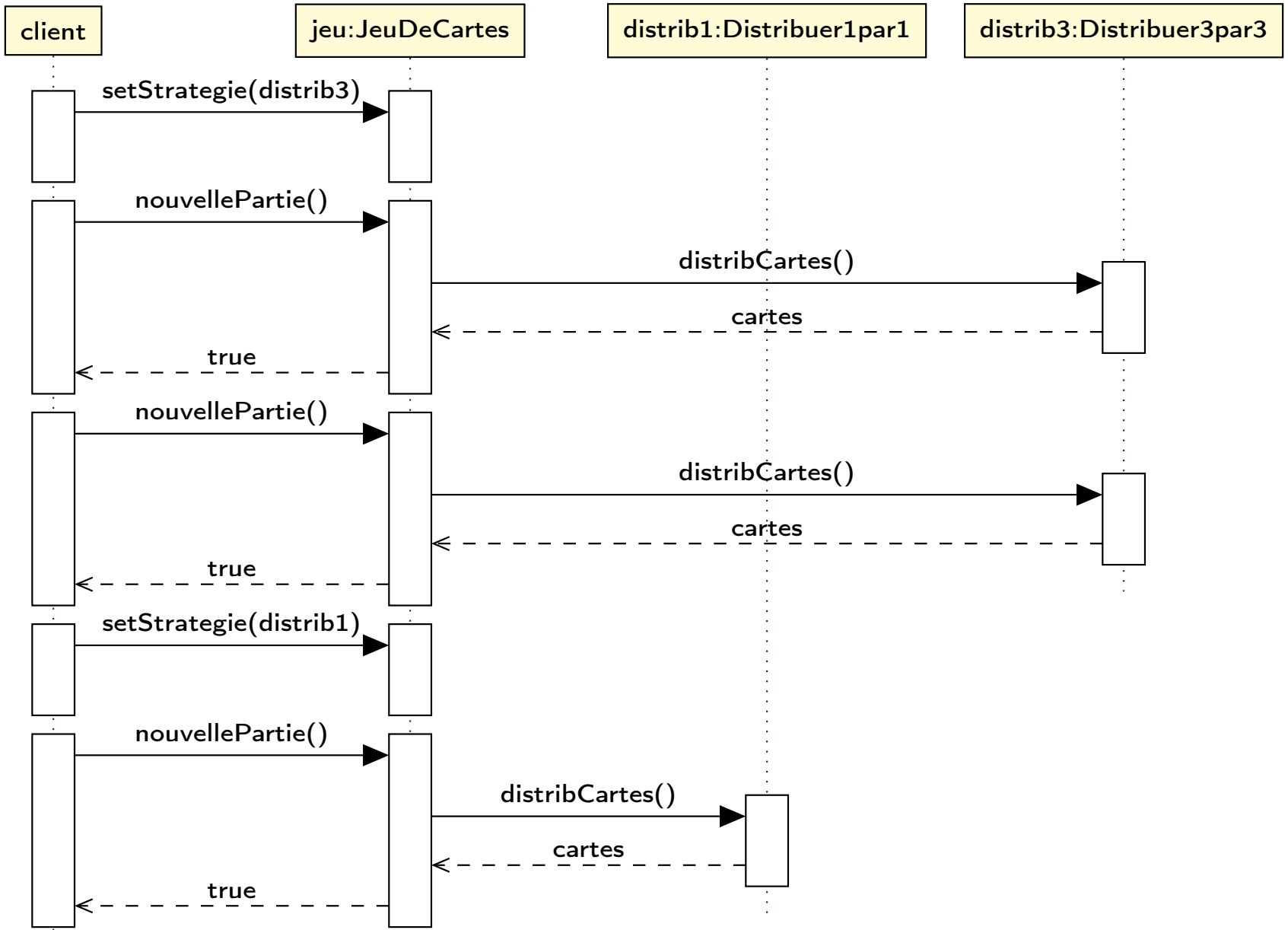
```java
class Composant {
    private Strategie strategie;

    public Composant(Strategie strategie) {
        this.strategie = strategie;
    }

    public void setStrategie(Strategie strategie) {
        this.strategie = strategie;
    }

    public void traitement() {
        this.strategie.operation();
    }
}
```

« Stratégie » – Diagramme de séquence



CC BY

Patron de conception « Composite »

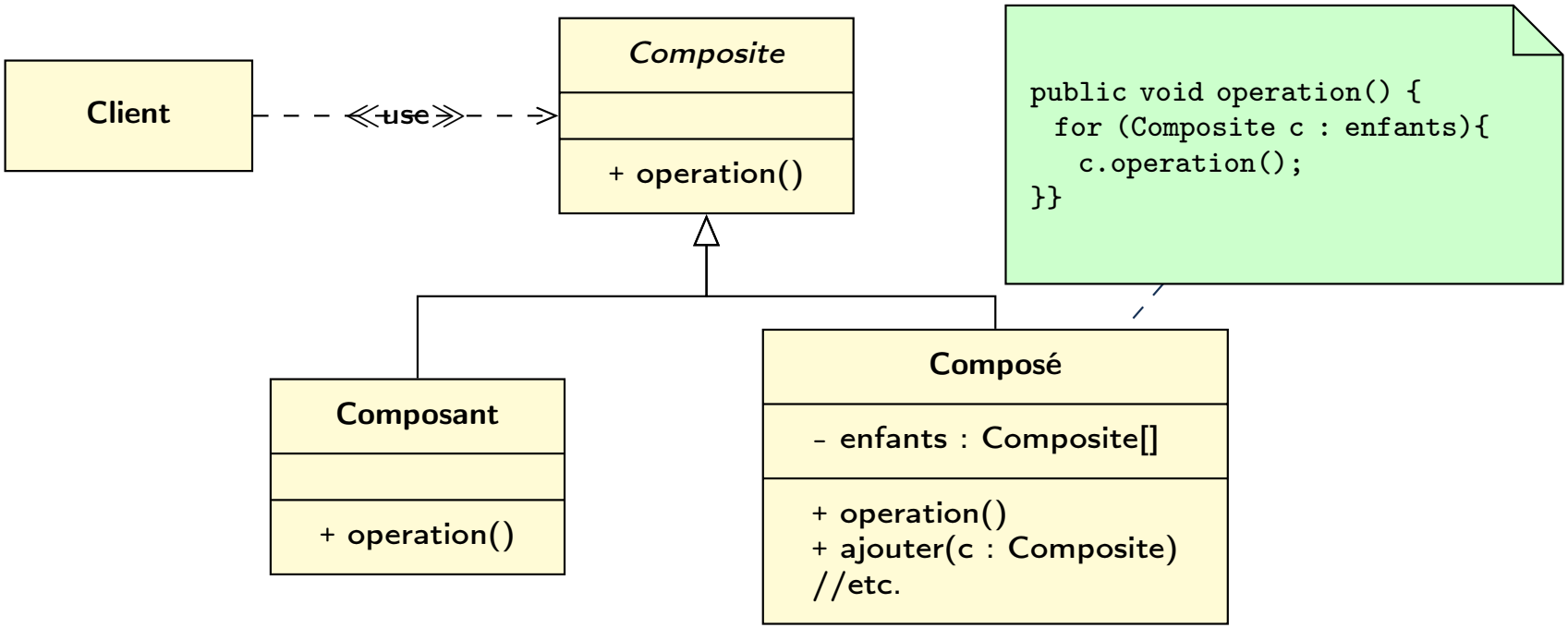
Permettre au client de traiter de la même façon des objets individuels et des groupes d'objets.

- **Problématique :** Traiter des hiérarchies de l'individu à l'ensemble. Traiter les constituants et et combinaisons de la même façon.
- **Solution :** Créer une classe abstraite spécifiant le comportement commun aux composés et aux composants
- **Exemples :** Arbre DOM (html/xml), ensembles, propositions logiques, etc. arbres syntaxiques

(rôle = structure ; portée = objet)

CC BY

« Composite » – Diagramme de classe



CC BY

« Composite » – En Java

```java
public abstract class Formule {
   public abstract boolean valeur();
}
```

```java
public class Prop extends Formule {
   private boolean valeur;

   public Prop(boolean valeur) {
      this.valeur = valeur;
   }
   public boolean valeur() {
      return this.valeur;
   }
}
```

```java
public class Et extends Formule {
   private Formule a;
   private Formule b;

   public Et(Formule a, Formule b) {
      this.a = a;
      this.b = b;
   }
   public boolean valeur() {
      return a.valeur() && b.valeur();
   }
}
```

```java
public class Neg extends Formule {
   private Formule a;

   public Neg(Formule a) {
      this.a = a;
   }

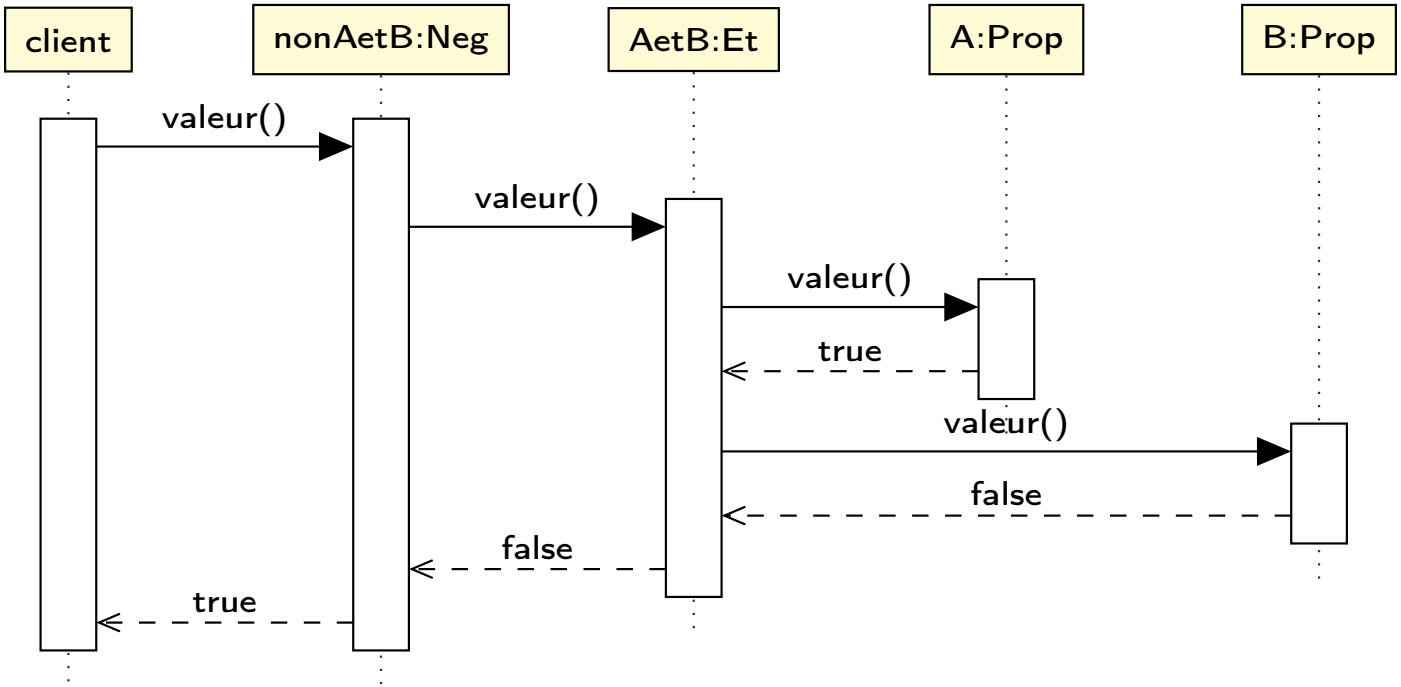
   public boolean valeur() {
      return ! a.valeur();
   }
}
```

```java
// Quelque part dans le code
Formule a = new Prop(true);
Formule b = new Prop(false);
Formule a_et_b = new Et(a,b);
Formule non_a_et_b = new Neg(a_et_b);

non_a_et_b.valeur();
```

CC BY

« Composite » – Diagramme de séquence




```
1  Formule a = new Prop(true);
2  Formule b = new Prop(false);
3  Formule a_et_b = new Et(a,b);
4  Formule non_a_et_b = new Neg(a_et_b);
5
6  non_a_et_b.valeur();
```

CC BY

Patron de conception « Itérateur »

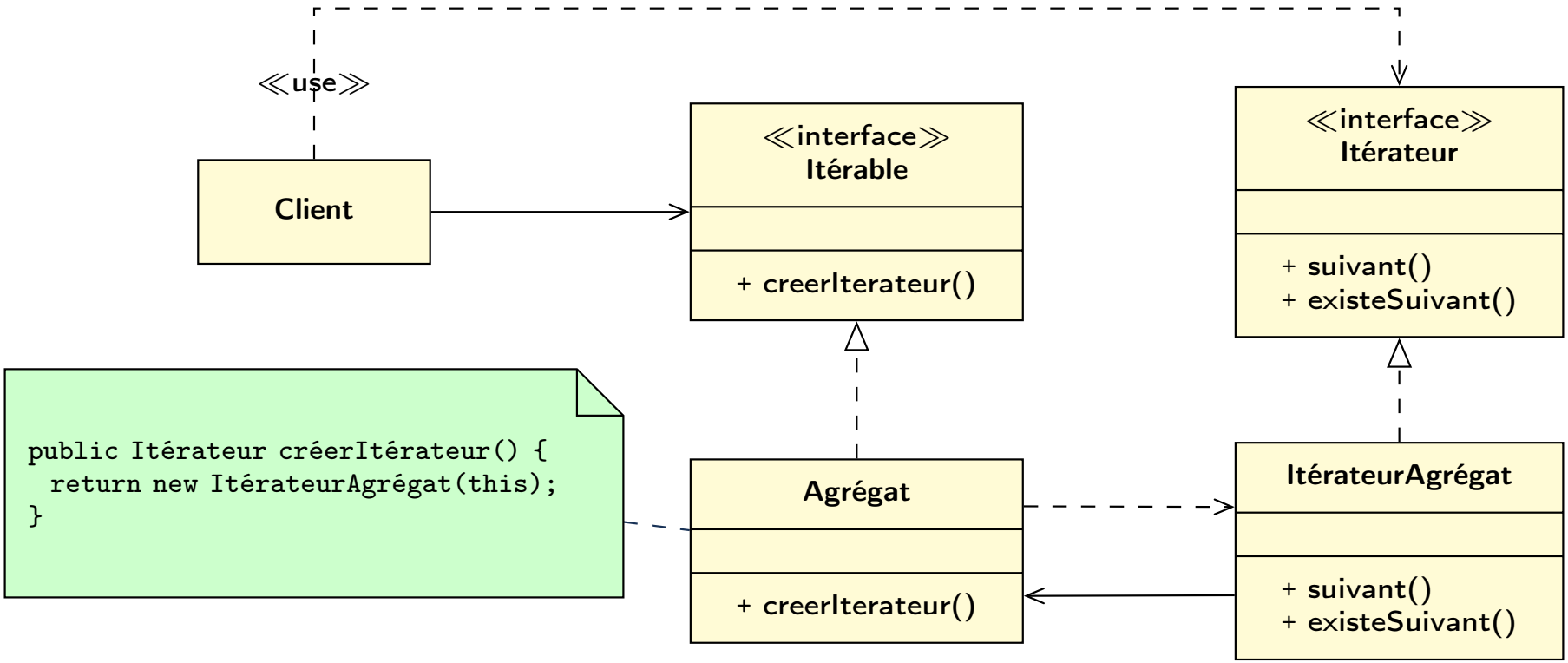
Séparer le parcours d'un agrégat et sa représentation

- **Problématique :** Permettre de parcourir une collection sans révéler sa structure interne.
  Gérer plusieurs parcours (en largeur d'abord, en profondeur, d'abord, etc.).
  Offrir une interface de parcours unique.
- **Solution :** Déléguer le parcours dans un objet annexe généré à la demande.
- **Exemples :** Parcours de liste, d'arbre, d'ensemble, etc.

(rôle = comportement ; portée = objet)

CC BY

« Itérateur » – Diagramme de classe




```
1  Iterateur it = agrégat.créerIterateur();
2  while (it.existeSuivant()) {
3      Elément e = it.suivant();
4      // traitement de l'élément e
5  }
```

CC BY

« Itérateur » – En Java

→ Implémenter l'interface `Iterable`. On dispose en plus du "foreach".

```java
public class Triplet<T> implements
      Iterable<T> {

  private T v1;
  private T v2;
  private T v3;

  public TripletInt(T v1, T v2, T v3) {
    this.v1 = v1;
    this.v2 = v2;
    this.v3 = v3;
  }
  @Override
  public Iterator<T> iterator() {
    return new IterTriplet<T>(this);
  }
}
```

```java
// Quelque part dans le code
Triplet<Integer> triplet
      = new Triplet<>(10,20,30);

for (int i : triplet) {
  System.out.print(i);
}
// Affichage : 10 20 30
```

```java
private class IterTriplet<T> implements
      Iterator<T> {

  private Triplet<T> triplet;
  private int curseur = 0;

  private IterTriplet(Triplet<T> t) {
    this.triplet = t;
  }
  @Override
  public boolean hasNext() {
    return curseur < 3;
  }
  @Override
  public T next() {
    curseur++;
    switch (curseur) {
    case 1:
      return this.triplet.v1;
    case 2:
      return this.triplet.v2;
    default:
      return this.triplet.v3;
    }
  }
}
```

CC BY

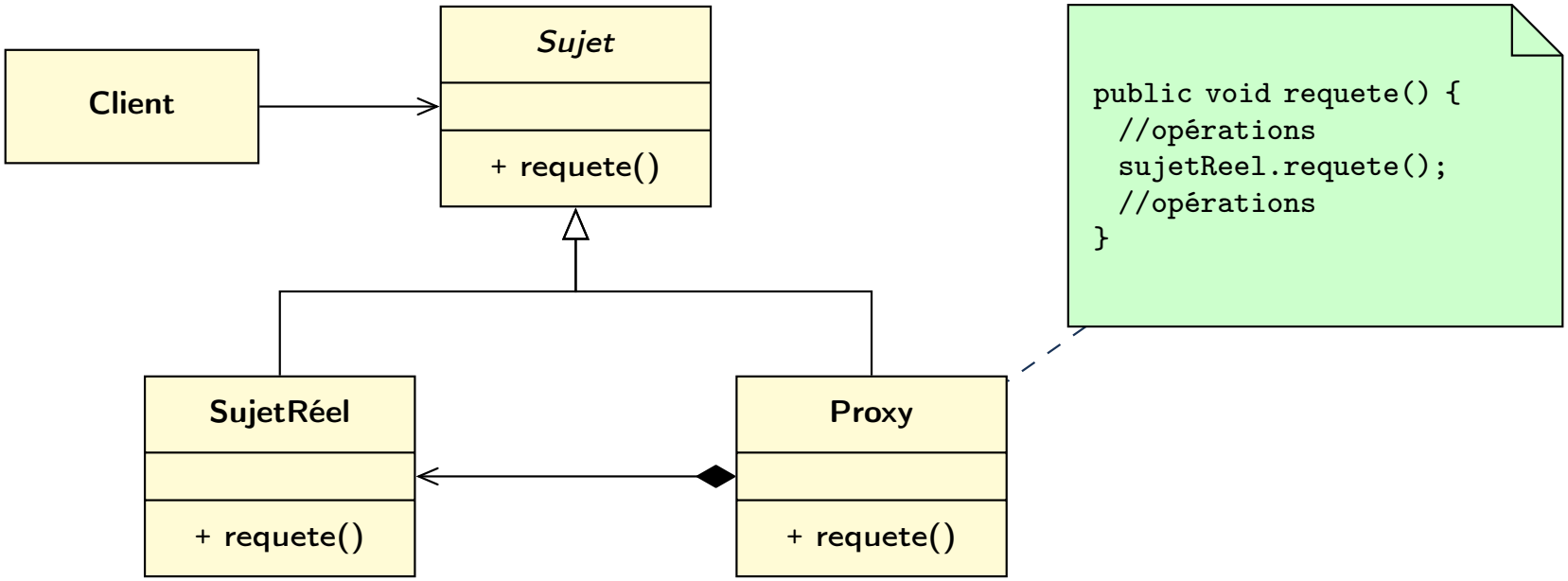
Patron de conception « Proxy »

Fournir un remplaçant d'un objet pour en contrôler l'accès.

- **Problématique :** On souhaite contrôler l'accès pour différer sa création, son initialisation, etc.
- **Solution :** Créer un objet tiers qui instancie l'objet au moment voulu et rediriger les requêtes vers cet objet tiers.
- **Exemples :** Contrôle des droits d'accès, verrous, gestion des pointeurs (garbage collector), etc.

(rôle = structure ; portée = objet)

« Proxy » – Diagramme de classe



CC BY

Patron de conception « Décorateur »

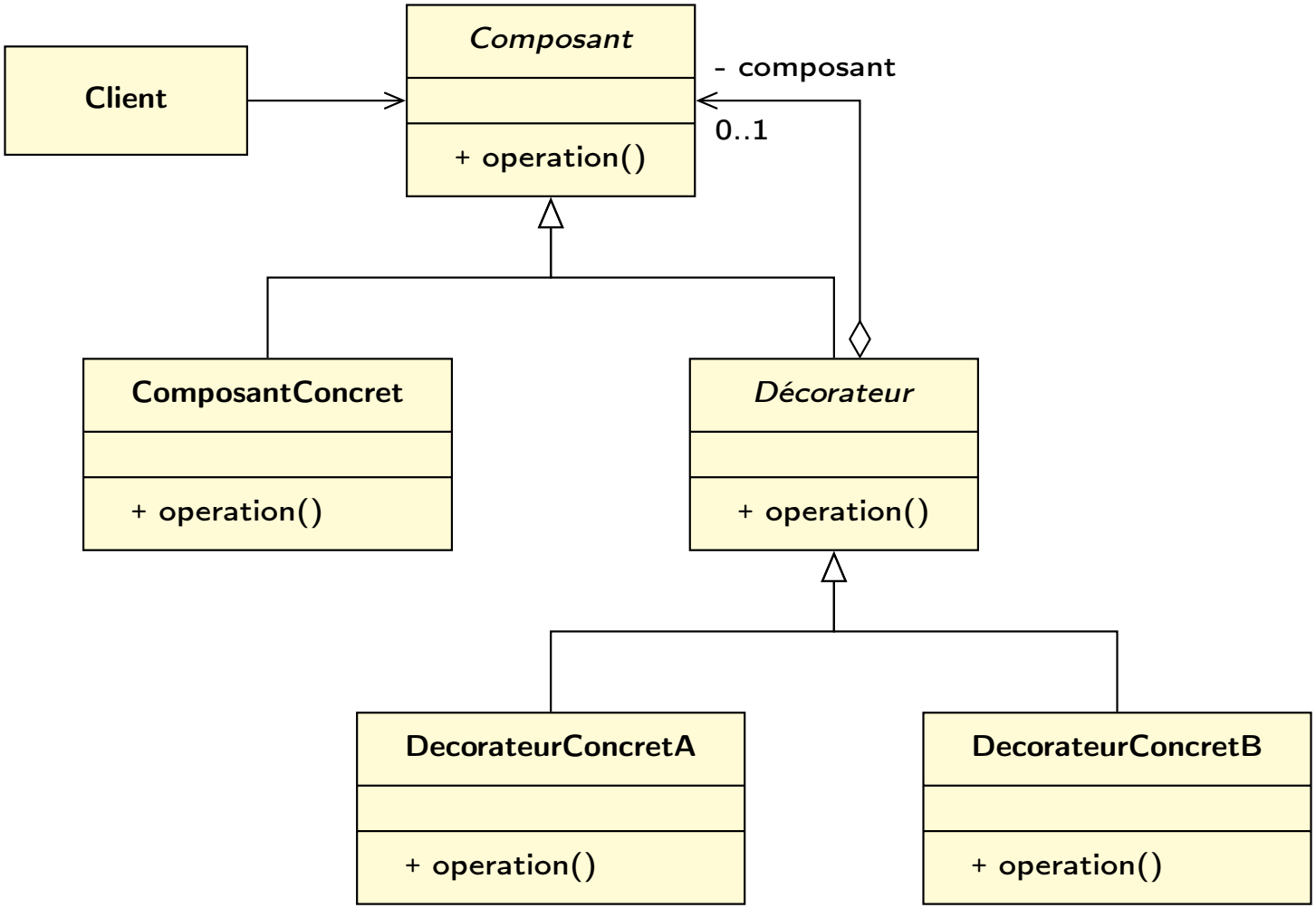
Attacher dynamiquement des responsabilité supplémentaires à un objet.

- **Problématique :** Améliorer la fonction de base d'un objet en lui ajoutant des opérations supplémentaires
- **Solution :** Chaîner dynamiquement les extensions de la fonctionnalité.
- **Exemples :** Fenêtre d'affichage + avec barre de défilement horizontale + avec barre de défilement verticale

(rôle = structure ; portée = objet)

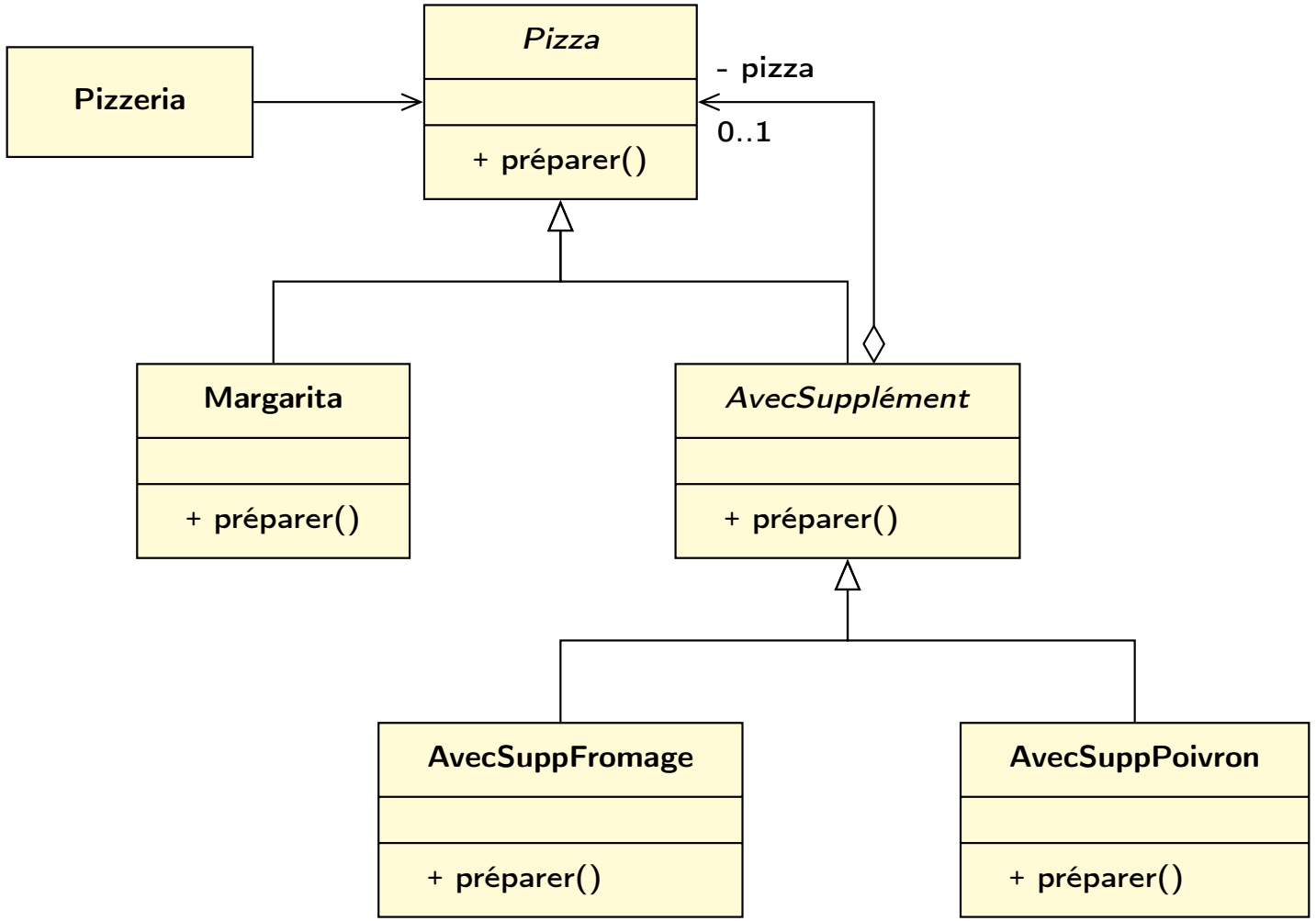
CC BY

« Décorateur » – Diagramme de classe



CC BY

« Décorateur » – Diagramme de classe



CC BY

« Décorateur » – En Java

```java
public abstract class Pizza {
  public abstract void preparer();
}
```

```java
public abstract class Margherita
      extends Pizza {
  public void preparer() {
    etalerPate();
    ajouterTomate();
    ajouterMozzarella();
    ajouterBasilic();
  }
}
```

```java
public abstract class AvecSupplement
      extends Pizza {

  private Pizza pizza;

  public AvecSupplement(Pizza pizza) {
    this.pizza = pizza;
  }

  public void preparer() {
    this.pizza.preparer();
  }
}
```

```java
public class AvecSuppFromage extends
      AvecSupplement {

  public void preparer() {
    super.preparer();
    ajouterFromage();
  }
}
```

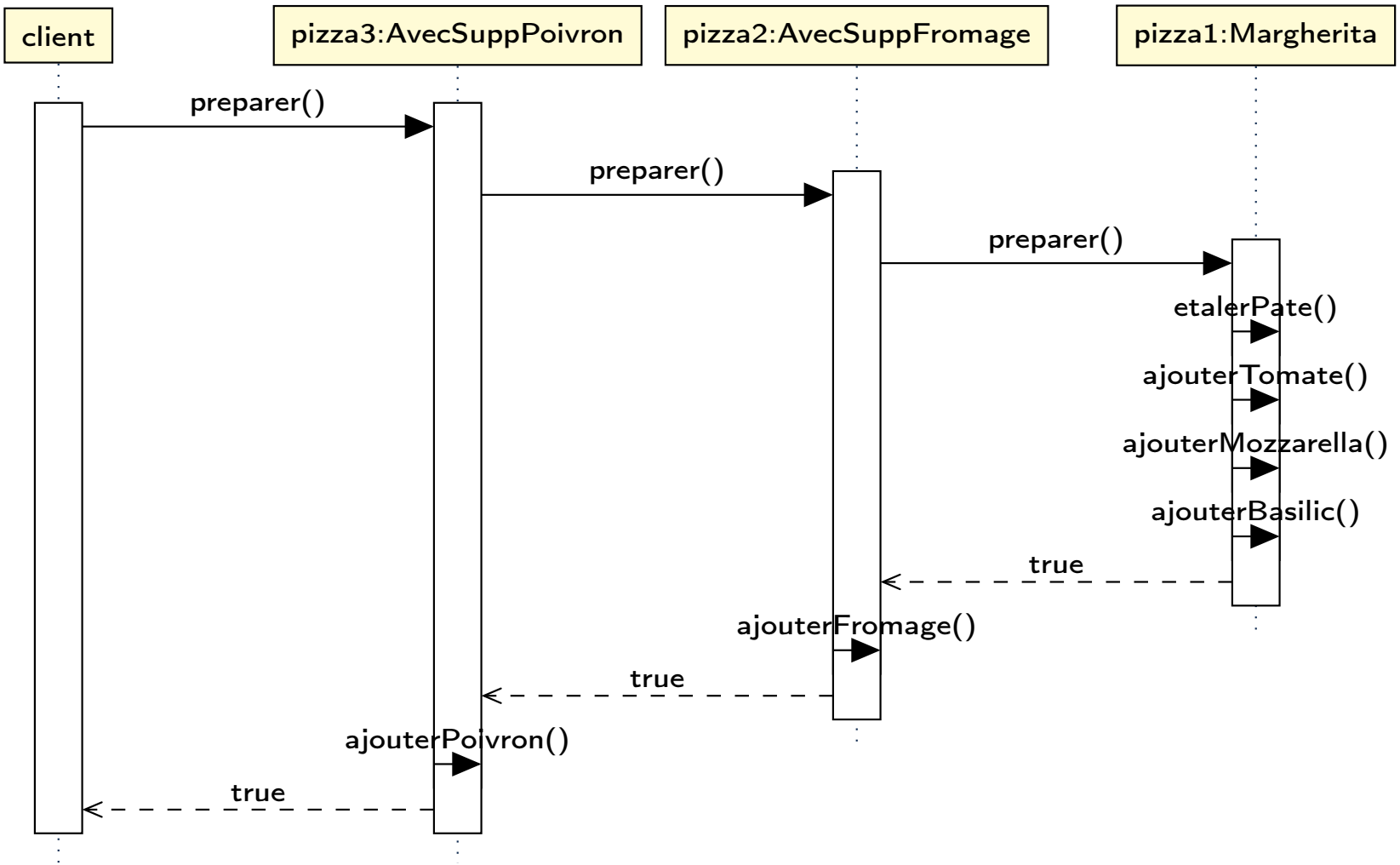
```java
public class AvecSuppPoivron extends
      AvecSupplement {

  public void preparer() {
    super.preparer();
    ajouterPoivron();
  }
}
```

```java
// Quelque part dans le code
Pizza pizza1 = new Margherita();
Pizza pizza2 = AvecSuppFromage(pizza1);
Pizza pizza3 = AvecSuppPoivron(pizza2);
pizza3.preparer();
```

CC BY

« Décorateur » – Diagramme de séquence



CC BY

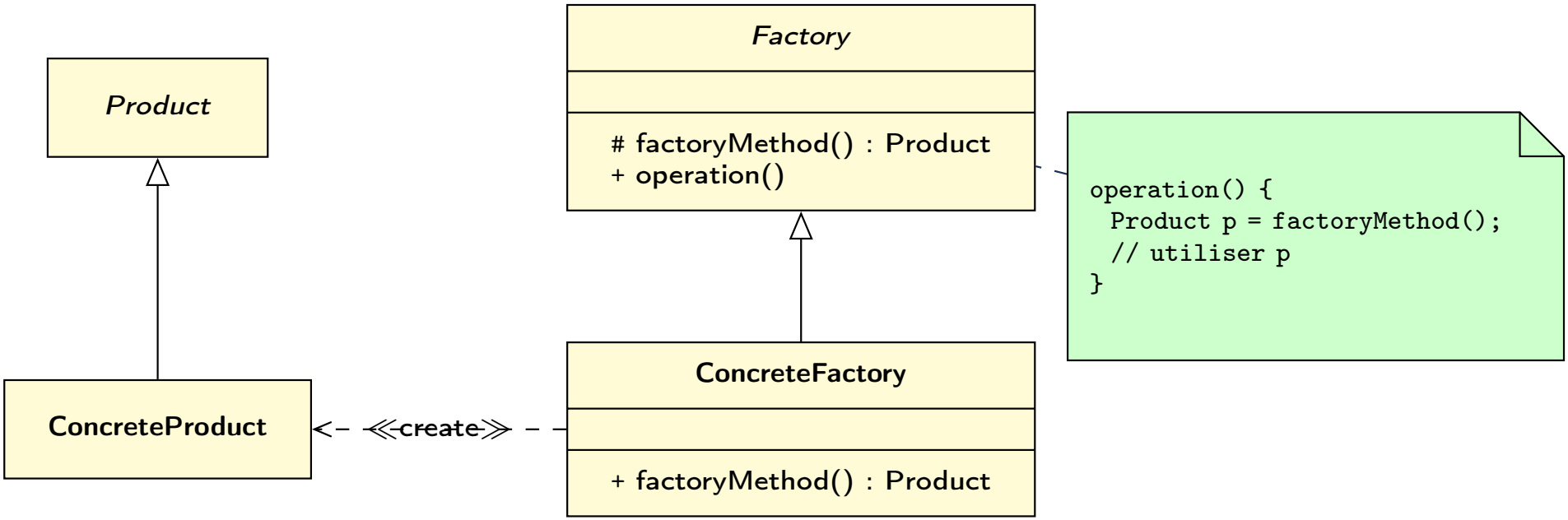
Patron de conception « Fabrique simple »

Définir une interface de création d'objets en laissant à ses sous-classes le soin de choisir la classe à instancier.

- **Problématique :** Standardiser le modèle architectural tout en permettant aux clients de définir leurs propres objets.
- **Solution :** Encapsuler la création d'instances dans une classe dédiée
- **Exemples :** Création d'objets similaires mais différents (éléments d'interface graphique, blocs, html, pizzas, etc.)

(rôle = création ; portée = classe)

« Fabrique simple » – Diagramme de classe



CC BY

« Fabrique simple » – En Java

```java
public abstract class Pizza {

  private Ingredient[] ingredients;

  public Pizza(Ingredient[] ingredients) {
    this.ingredients = ingredients;
  }

  public abstract void postprocessing();

  public Ingredient[] getIngredients() {
    return this.ingredients;
  }
}
```

```java
public abstract class PizzaFactory {

  protected abstract Pizza factoryMethod();

  public void preparer() {
    Pizza p = factoryMethod():
    etalerPate(p);
    for (Ingredient i : pizza.getIngredients()) {
      ajouterIngredient(p, i);
    }
    pizza.postprocessing();
    cuire(pizza);
    mettreEnBoite(pizza);
  }
}
```

CC BY

« Fabrique simple » – En Java

```java
public Margherita extends Pizza {
  public Margherita() {
    super({tomate,mozza,basilic});
  }
  public void postprocessing() {}
}
```

```java
public final class MargheritaFactory
     extends Factory {
  @Override
  protected Pizza factoryMethod() {
    return new Margherita();
  }
}
```

```java
public Calzone extends Pizza {
  public Calzone() {
    super({tomate,mozza,jambon});
  }
  public void postprocessing() {
    this.casserOeuf();
    this.replierPate();
  }
}
```

```java
public final class CalzoneFactory
     extends Factory {
  @Override
  protected Pizza factoryMethod() {
    return new Calzone();
  }
}
```

```java
// Quelque part dans le code
PizzaFactory margheritaFactory = new MargheritaFactory();
PizzaFactory calzoneFactory = new CalzoneFactory();

// Préparer une margherita et deux calzones
margheritaFactory.preparer();
calzoneFactory.preparer();
calzoneFactory.preparer();
```

CC BY

Patron de conception « Observateur »

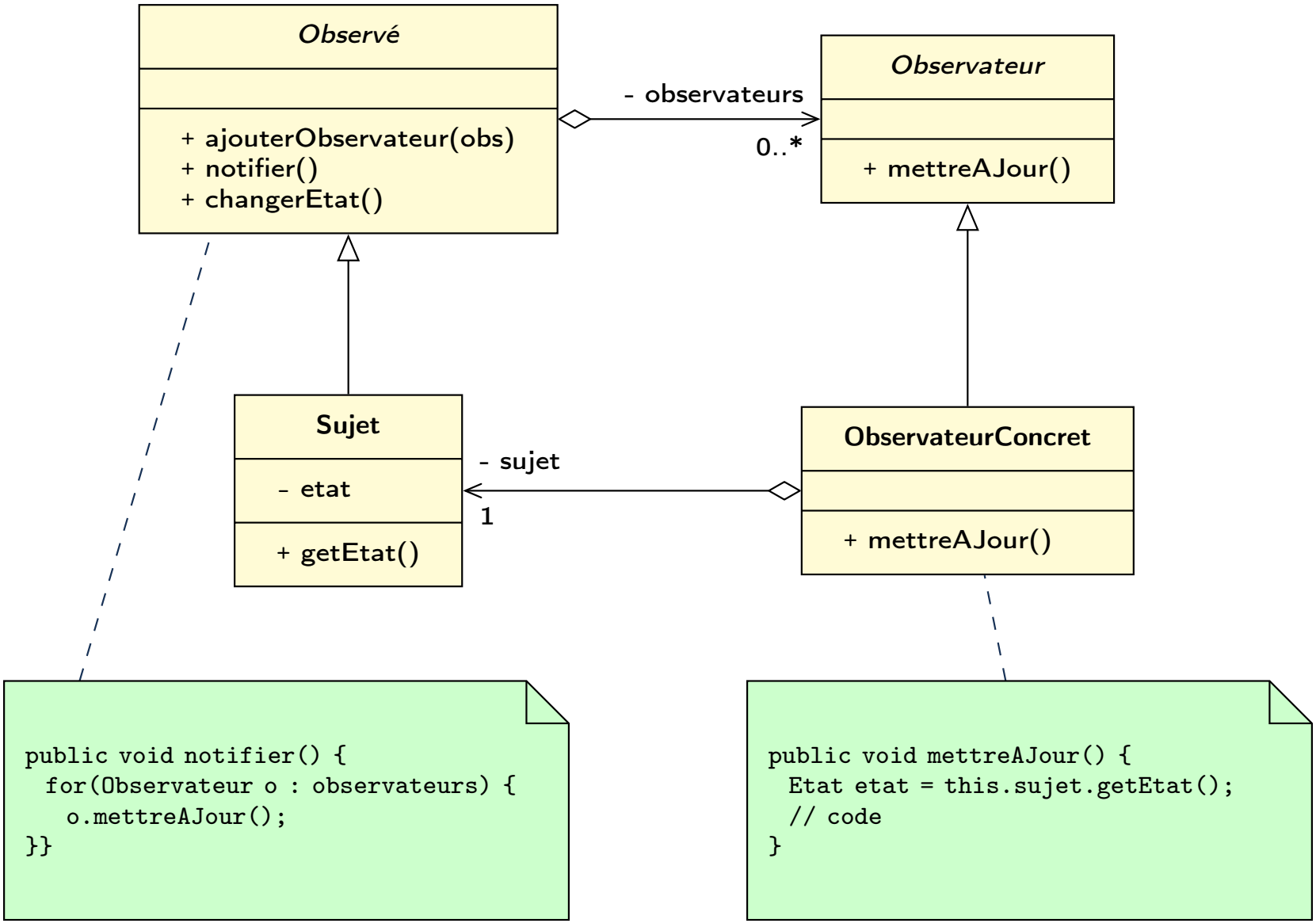
Définir une dépendance 1-n de manière à ce que les n observateurs soient informés du changement d'état de l'objet.

- **Problématique :** Informer plusieurs objets (non connus à l'avance) qu'un changement a eu lieu.
- **Solution :** Notifier les observateurs qui se seront enregistrés auprès de l'observé.
- **Exemples :** Dans le tableur OOCalc, le changement d'une valeur modifie toutes les valeurs des cases qui y font référence (et des graphiques, etc.)

(rôle = comportement ; portée = objet)

CC BY

« Observateur » – Diagramme de classe



CC BY

« Observateur » – En Java

Java propose les interfaces `Observer` et `Observable`...
mais elles sont dépréciées depuis Java 9 !

On préfèrera utiliser les différentes interfaces `XXXListener`.

```java
public class Diffuseur {
        private String actu;
        private PropertyChangeSupport support;

        public Diffuseur() {
                support = new PropertyChangeSupport(this);
        }

        public void addListener(PropertyChangeListener obs) {
                support.addPropertyChangeListener(obs);
        }
        public void removeListener(PropertyChangeListener obs) {
                support.removePropertyChangeListener(obs);
        }
        public void setActu(String nouvelleActu) {
                support.firePropertyChange("actu", this.actu, nouvelleActu);
                this.actu = nouvelleActu;
        }
}
```

CC BY

« Observateur » – En Java

```java
public class Follower implements PropertyChangeListener {
        private String nom;

        public Follower(String nom) {
                this.nom = nom;
        }
        public void propertyChange(PropertyChangeEvent evt) {
                System.out.println(nom + "␣:␣L'actu␣a␣changé␣!␣C'était␣" + evt.
                     getOldValue() + "␣et␣c'est␣devenu␣" + evt.getNewValue());
        }
}
```

```java
// Quelque part dans le code
Diffuseur d = new Diffuseur();

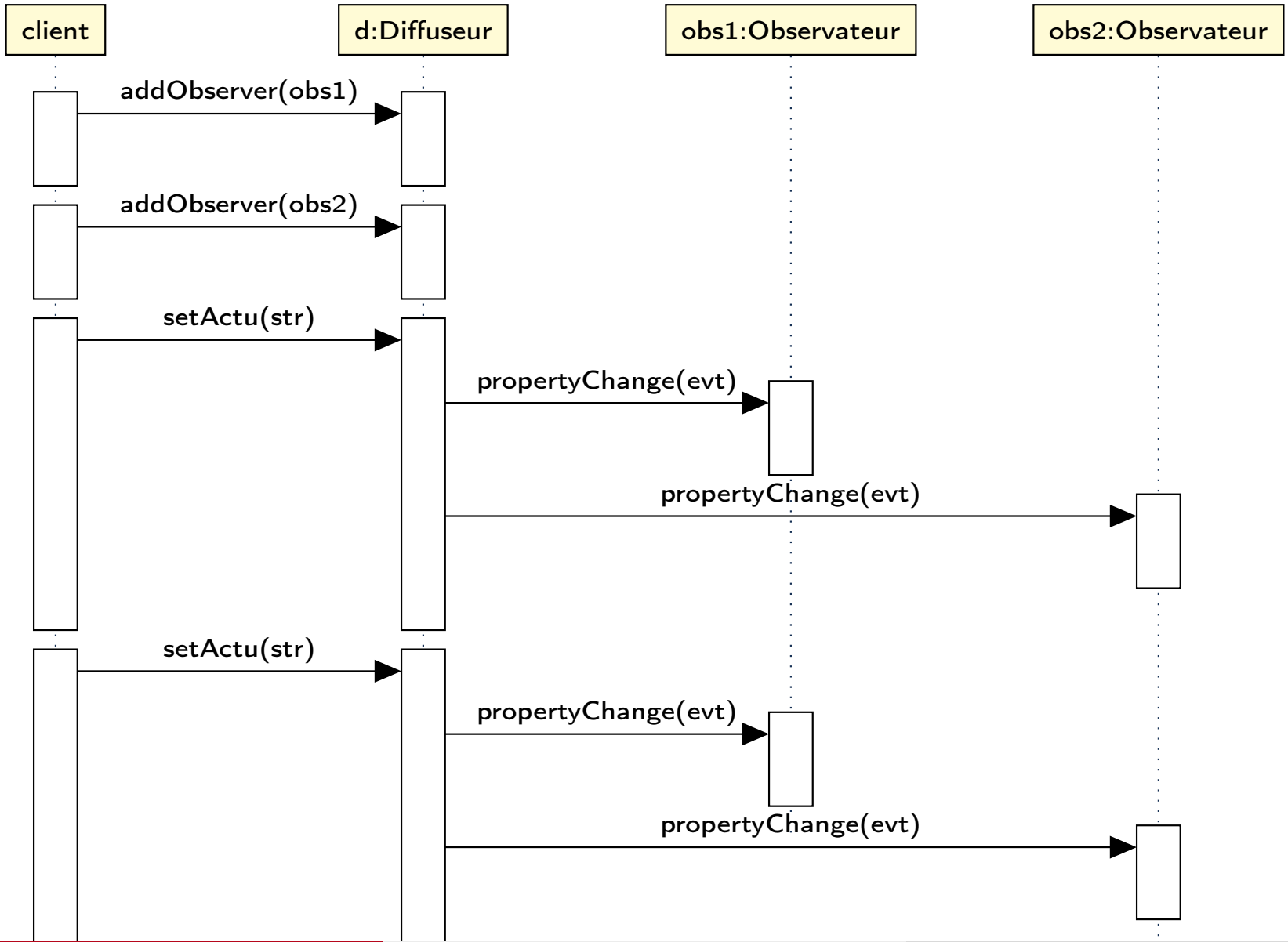
Follower obs1 = new Follower("Alice");
d.addListener(obs1);

Follower obs2 = new Follower("Bob");
d.addListener(obs2);

d.setActu("J'aime␣!");
d.setActu("Je␣like␣!");

// Affichage :
// Alice : L'actu a changé ! C'était null et c'est devenu J'aime !
// Bob :   L'actu a changé ! C'était null et c'est devenu J'aime !
// Alice : L'actu a changé ! C'était J'aime ! et c'est devenu Je like !
// Bob :   L'actu a changé ! C'était J'aime ! et c'est devenu Je like !
```

« Observateur » – Diagramme de séquence



CC BY

Références

Le livre de référence :
Design Patterns : Elements of Reusable Object-Oriented Software.
Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994).
Inde : Pearson Education.

En accès libre sur WikiBooks :
Les patrons de conceptions du GoF

Beaucoup de ressources en ligne avec les mots clés “patrons de conception”, “design pattern”, “Gang of Four”.

CC BY